

# Simulation de collision de particules avec potentiel de Lennard-Jones

Faical EL GOUIJ

2026-03-24

## 1. Introduction

L'objectif de ce TP est de mettre en place une simulation de particules en interaction via un potentiel de Lennard-Jones, tout en rendant le calcul scalable grâce à un découpage spatial.

Le potentiel de Lennard-Jones est donné par :

$$U(r) = 4\varepsilon \left[ \left( \frac{\sigma}{r} \right)^{12} - \left( \frac{\sigma}{r} \right)^6 \right]$$

La force associée est :

$$F = 24\varepsilon \cdot \frac{1}{r^2} \cdot \left( \left( \frac{\sigma}{r} \right)^6 \left( 1 - 2 \left( \frac{\sigma}{r} \right)^6 \right) \right) \cdot \vec{r}$$

Comme indiqué dans l'énoncé :contentReferenceoacite:1, un rayon de coupure  $r_{cut}$  est introduit afin de limiter le coût des calculs.

## 2. Architecture du projet

### 2.1 Choix des classes

Le projet est structuré autour de 4 classes principales :

- **vecteur** : représente un vecteur 3D
- **particule** : contient position, vitesse, force
- **cellule** : contient un ensemble de particules et ses voisins
- **univers** : gère l'ensemble de la simulation

Ce découpage permet :

- une bonne séparation des responsabilités
- une extensibilité (autres forces, autres dimensions)

## 2.2 Choix d'un vecteur 3D unique

Le choix a été fait d'utiliser un **vecteur toujours en 3D**, même pour des simulations 1D ou 2D.

Cela évite :

- la duplication du code
- la gestion de plusieurs types de vecteurs

## 3. Gestion des dimensions (1D, 2D, 3D)

Le code supporte dynamiquement :

- 1D
- 2D
- 3D

Cela est géré dans `univers::ajoute_particule` :

- en 1D : on annule y et z
- en 2D : on annule z
- en 3D : rien n'est modifié

Cela permet d'éviter une projection coûteuse à chaque calcul.

### Limitation :

Cette stratégie suppose que **les particules ne sont pas ajoutées dynamiquement après initialisation**. Sinon, il faudrait reprojetter les coordonnées.

## 4. Découpage spatial (cellules)

Comme demandé dans la consigne :contentReferenceoaicite:2, l'espace est découpé en cellules.

### 4.1 Construction

Le nombre de cellules par dimension est :

$$n_{cd} = \left\lceil \frac{L_d}{r_{cut}} \right\rceil$$

Chaque cellule contient :

- une liste de particules
- une liste de cellules voisines

Nombre de voisins :

- 1D : 3
- 2D : 9
- 3D : 27

## 4.2 Mise à jour des cellules

À chaque pas de temps :

- les particules sont replacées dans les cellules
- cela permet de limiter les interactions

## 5. Calcul des forces

### 5.1 Optimisations mises en place

#### 1. Troisième loi de Newton

On exploite :

$$F_{ij} = -F_{ji}$$

Donc chaque interaction est calculée une seule fois :

```
pi->ajouterForce(...)
pj->ajouterForce(-...)
```

#### 2. Utilisation de $r^2$ au lieu de $r$

```
double dist2 = dx*dx + dy*dy + dz*dz;
```

#### 3. Éviter les pow

```
double sr2 = sigma2 / dist2;
double sr6 = sr2 * sr2 * sr2;
double sr12 = sr6 * sr6;
```

#### 4. Pas de vecteurs temporaires

Au lieu de:

```
vecteur r = posj - posi;
```

on utilise directement:

```
double dx, dy, dz;
```

Pour éviter la construction d'objets complexes ce qui peut occuper plus de mémoire.

## 6. Intégration temporelle

L'intégration temporelle est un élément central de la simulation, car elle détermine l'évolution des particules dans le temps à partir des forces calculées.

### 6.1 Problème initial : méthode d'Euler

Une première approche a consisté à utiliser une méthode de type Euler pour mettre à jour les positions et vitesses des particules :

$$v(t + dt) = v(t) + \frac{F(t)}{m} dt$$

$$x(t + dt) = x(t) + v(t) dt$$

Cette méthode est simple à implémenter, mais elle présente plusieurs défauts importants dans le cadre de systèmes physiques :

- elle n'est pas symplectique (elle ne conserve pas l'énergie du système) ;
- elle accumule des erreurs numériques au cours du temps ;
- elle est particulièrement instable pour des interactions fortement non linéaires comme le potentiel de Lennard-Jones.

Dans nos simulations, cela s'est traduit par :

- une divergence rapide du système ;
- des vitesses qui deviennent irréalistes ;
- un comportement physique incohérent (explosion du système).

### 6.2 Solution retenue : schéma de Störmer-Verlet

Afin de résoudre ces problèmes, l'intégration a été remplacée par le schéma de **Störmer-Verlet**, déjà introduit lors du TP2.

Ce schéma est basé sur deux étapes :

1. Mise à jour des positions :

$$x(t + dt) = x(t) + v(t) dt + \frac{1}{2} \frac{F(t)}{m} dt^2$$

2. Mise à jour des vitesses :

$$v(t + dt) = v(t) + \frac{1}{2} \left( \frac{F(t)}{m} + \frac{F(t + dt)}{m} \right) dt$$

Dans le code, cela correspond aux fonctions :

```
avance_position_verlet(dt);  
calcule_forces();  
avance_vitesse_verlet(dt);
```

### 6.3 Avantages du schéma Verlet

Le schéma de Störmer-Verlet présente plusieurs avantages importants pour la simulation de systèmes physiques : - **Stabilité numérique** : il permet d'éviter la divergence du système même sur de longues simulations ; - **Conservation de l'énergie** : contrairement à Euler, il conserve mieux l'énergie totale du système ; - **Méthode symplectique** : adaptée aux systèmes hamiltoniens comme les interactions de particules ; - **Bonne précision** pour un coût de calcul raisonnable.

Dans notre cas, le passage d'Euler à Verlet a permis de corriger les comportements non physiques observés (explosion du système).

### 7. Découpage en cellules

L'optimisation principale du calcul de force repose sur le **maillage de l'espace en cellules**.

Comme demandé dans l'énoncé, l'Univers est découpé en une grille tensorielle de cellules, dont le nombre dans chaque direction est donné par :

$$n_{cd} = \left\lfloor \frac{L_d}{r_{cut}} \right\rfloor$$

Chaque cellule a donc une taille de l'ordre de  $r_{cut}$  dans chaque direction. Cette construction est précisément celle suggérée par la consigne, qui demande aussi que chaque cellule connaisse ses voisines : 3 en 1D, 9 en 2D, 27 en 3D.

**Idée générale** Sans découpage spatial, pour calculer les interactions entre particules, il faudrait comparer chaque particule à toutes les autres.

Pour  $N$  particules, cela conduit à un coût en  $O(N^2)$ , ce qui devient rapidement trop coûteux.

Or, avec le potentiel de Lennard-Jones tronqué, on néglige toute interaction au-delà du rayon de coupure  $r_{cut}$ .

Il n'est donc pas utile de tester toutes les particules du système : il suffit de tester celles qui sont **assez proches**.

**Pourquoi les cellules permettent de réduire le calcul** Le principe est le suivant :

- on associe chaque particule à une cellule à partir de sa position ;
- pour une particule donnée, on ne cherche des interactions que :
  - dans sa cellule,
  - ou dans les cellules voisines.

Cela est suffisant parce que :

- la taille d'une cellule est construite à partir de  $r_{cut}$  ;
- deux particules séparées par plus d'une cellule dans au moins une direction sont nécessairement trop éloignées pour interagir.

Autrement dit, si une particule se trouve dans une cellule donnée, seules les particules situées dans cette cellule ou dans une cellule adjacente peuvent être à une distance inférieure à  $r_{cut}$ .

**Cas 1D, 2D et 3D** Le nombre de cellules voisines dépend de la dimension :

- en **1D**, une cellule a au plus 3 cellules pertinentes : elle-même, la cellule de gauche et celle de droite ;
- en **2D**, une cellule a au plus 9 cellules pertinentes : elle-même et les 8 cellules autour ;
- en **3D**, une cellule a au plus 27 cellules pertinentes : elle-même et les 26 cellules voisines.

C'est exactement ce qui est construit dans `initialise_cellules()`, où les voisins sont précalculés une fois pour toutes au début.

**Implémentation dans le code** Dans l'implémentation, l'Univers contient :

- un vecteur de cellules ;
- pour chaque cellule, une liste de particules ;
- pour chaque cellule, une liste de pointeurs vers ses cellules voisines.

À chaque pas de temps, les particules sont replacées dans les bonnes cellules grâce à :

```
vide_cellules()
place_particules_dans_cellules()
place_particule_dans_cellule(...)
```

L'indice logique d'une cellule est calculé par `indice_cellule(...)`, avec une formule différente selon la dimension.

**Conséquence sur le calcul des forces** Dans `calcule_forces()`, on ne parcourt donc pas toutes les particules du système entre elles.

On parcourt :

1. une cellule `c`,
2. les particules contenues dans `c`,
3. puis seulement les cellules voisines de `c`.

On distingue ensuite deux cas :

- **interaction à l'intérieur de la même cellule** : on fait une double boucle sur les particules de cette cellule ;
- **interaction entre deux cellules voisines distinctes** : on fait une boucle sur les particules de la cellule courante et une boucle sur celles de la cellule voisine.

De plus, pour éviter de compter deux fois la même interaction, on ignore les cellules voisines déjà traitées à l'aide du test sur les adresses :

```
if (v < &c) continue;
```

Le découpage spatial constitue une amélioration majeure pour la complexité.

Les limites observées ouvrent la voie à des optimisations avancées.

## 6.3 Avantages du schéma Verlet

Le schéma de Störmer-Verlet présente plusieurs avantages importants pour la simulation de systèmes physiques :

- **Stabilité numérique** : il permet d'éviter la divergence du système même sur de longues simulations ;
- **Conservation de l'énergie** : contrairement à Euler, il conserve mieux l'énergie totale du système ;
- **Méthode symplectique** : adaptée aux systèmes hamiltoniens comme les interactions de particules ;
- **Bonne précision** pour un coût de calcul raisonnable.

Dans notre cas, le passage d'Euler à Verlet a permis de corriger les comportements non physiques observés (explosion du système).

---

## 8. Simulation réalisée

### 8.1 Configuration

La simulation implémentée correspond à celle demandée dans l'énoncé :

- un carré de **40 × 40 particules** (type 1)
- un rectangle de **160 × 40 particules** (type 2)

La distance initiale entre particules est :

$$r = 2^{1/6}$$

Cette valeur correspond au minimum du potentiel de Lennard-Jones, ce qui permet d'obtenir un état initial stable.

Les conditions initiales sont :

- les particules du carré possèdent une vitesse initiale verticale
- les particules du rectangle sont initialement au repos

### 8.2 Résultat de la simulation

Au cours de la simulation, on observe :

- une **collision progressive** entre les deux ensembles de particules ;
- une **déformation du carré** lors de l'impact ;
- une **dispersion des particules** après la collision.

Le comportement observé est cohérent avec la physique du potentiel de Lennard-Jones :

- répulsion forte à courte distance ;
  - attraction à moyenne distance.
-

## 9. Visualisation

Deux modes de visualisation ont été implémentés.

### 9.1 Mode Python

Dans ce mode :

- les données sont exportées dans un fichier `.txt`
- un script Python utilisant `matplotlib` est exécuté

Avantages :

- simple à mettre en place
- utile pour le debug

Inconvénients :

- limité pour de grandes simulations
- performances modestes

### 9.2 Mode ParaView (VTK)

Dans ce mode :

- les données sont exportées en fichiers `.vtk`
- un fichier `animation.vtk.series` est généré

Cela permet d'utiliser ParaView pour visualiser l'animation.

Avantages :

- visualisation fluide
- rendu graphique avancé
- possibilité d'analyse interactive

#### Setup nécessaire

Pour un lancement automatique :

1. installer ParaView
2. ajouter au PATH :

```
““bash export PATH=$PATH:/chemin/vers/paraview/bin
```